Natural Language Understanding: Assignment Report

Problem 1: Word2Vec & Problem 2: Character-Level RNNs

Instructor: Dr. Anand Mishra

Submitted by: Kanishk Singh Dayma (B23CS1024)

Contents

1 Part 1: Word2Vec Implementation **2**
1.1 Theoretical Background: Word2Vec Architecture 2
1.2 Task 1: Dataset Preparation 2
1.3 Task 2: Model Training 4
1.4 Task 3: Semantic Analysis 4
1.5 Task 4: Visualization 5

2 Part 2: Character-Level Name Generation using RNN Variants **7**
2.1 Task 0: Dataset 7
2.2 Task 1: Model Implementation & Architecture 7
2.2.1 1. Vanilla Recurrent Neural Network (RNN) 7
2.2.2 2. Bidirectional Long Short-Term Memory (BiLSTM) 7
2.2.3 3. RNN with Basic Attention Mechanism 7
2.3 Task 2: Quantitative Evaluation 8
2.4 Task 3: Qualitative Analysis 8
2.4.1 1. Vanilla RNN: The Best Performer 8
2.4.2 2. Attention RNN: Overcomplicating Local Tasks 8
2.4.3 3. Bidirectional LSTM: Structural Failure Mode (Data Leakage) 9
2.4.4 Generated names: 9

1 Part 1: Word2Vec Implementation

1.1 Theoretical Background: Word2Vec Architecture

Before detailing the implementation, it is essential to establish the theoretical foundation of the models used. Word2Vec is a family of shallow, two-layer neural networks trained to reconstruct the linguistic contexts of words, ultimately producing dense vector representations (embeddings) where semantically similar words are mapped to proximate points in a continuous geometric space.

In this assignment, two primary architectures were implemented from scratch:

- **Continuous Bag of Words (CBOW):** The CBOW model predicts a single target (center) word based on the context of the surrounding words. It takes the distributed representations of the context words, averages them, and passes them through a linear layer to predict the center word. CBOW is generally faster to train and mathematically smooths over distributional statistics, making it highly effective for frequent words.

- **Skip-gram:** The Skip-gram model inverts the CBOW objective. It takes a single target (center) word as input and attempts to predict the surrounding window of context words. Because it treats each context-target pair as a new observation, Skip-gram is computationally heavier but traditionally performs better with smaller datasets and provides superior representations for rare words.

Optimization: Negative Sampling

Calculating the standard softmax denominator across an entire vocabulary of over 16,000 words for every single training step is computationally prohibitive. To optimize this, both models were implemented using **Negative Sampling**. Instead of updating all vocabulary weights during backpropagation, the models transform the task into a binary classification problem using a sigmoid function: they maximize the dot product similarity for the true context word (positive sample) while minimizing it for a small subset ($k = 3$ or $k = 5$) of randomly drawn "noise" words (negative samples). This drastically reduces computational latency while maintaining high-quality embeddings.

1.2 Task 1: Dataset Preparation

The dataset was curated from three primary sources: local raw text files, academic and regulatory PDFs downloaded from the IIT Jodhpur website, and a custom web crawler configured to scrape up to 700 pages from the `iitj.ac.in` domain.

The preprocessing pipeline strictly adhered to the following steps:

1. **Data Extraction & Unicode Scrubbing**: Text was extracted such that one single text line, one entire PDF file, and one complete webpage each represented a single document. Hidden Unicode characters and PDF formatting artifacts were scrubbed.

2. **Normalization**: Academic degrees were normalized using regex (e.g., "b.tech" to "btech").

3. **Artifact Removal**: URLs, emails, numbers, and all non-alphabetic punctuation were removed.

4. **Tokenization & Filtering**: The text was tokenized. NLTK English stopwords and custom academic noise words (e.g., "shall", "hence", "per") were removed. Words containing two characters or fewer were dropped, explicitly protecting vital acronyms (e.g., ug, pg, cse).

5. **Lemmatization**: A WordNet Lemmatizer was applied to convert words to their base dictionary forms.

Dataset Statistics

- **Total Documents:** 675
- **Total Tokens:** 296,425
- **Vocabulary Size:** 16,216

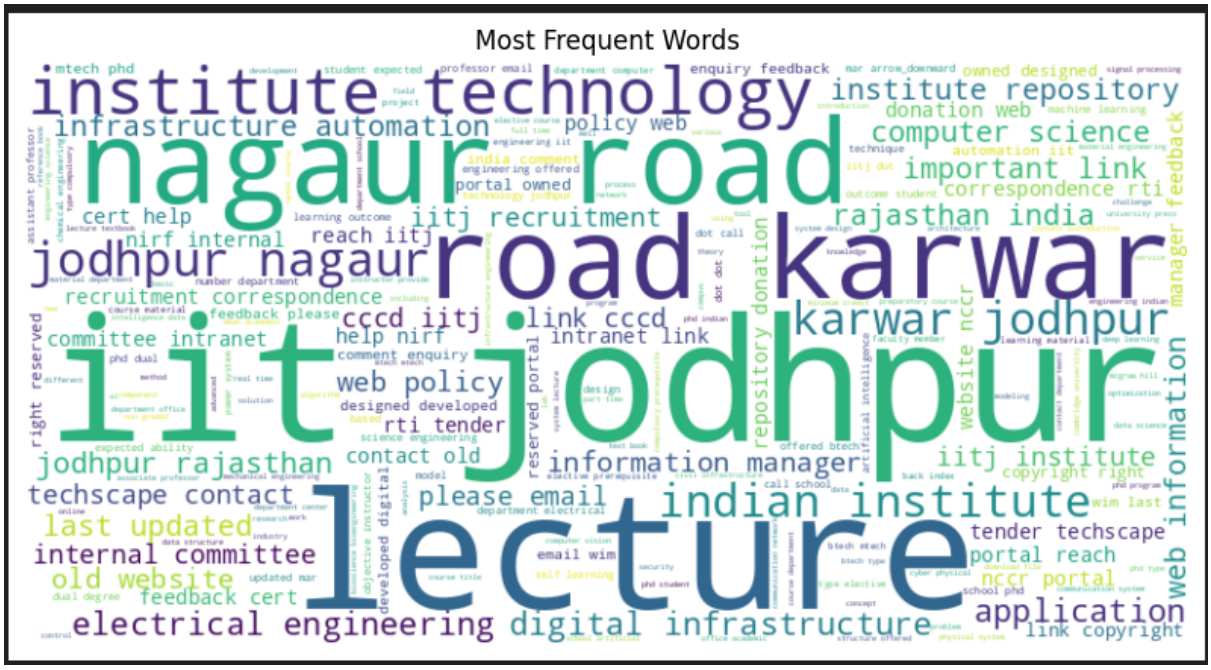

Figure 1: Word Cloud illustrating the most frequent words in the IITJ corpus.

1.3 Task 2: Model Training

Both CBOW and Skip-gram architectures were trained via a systematic grid search to evaluate the impact of different hyperparameter configurations. The search spanned embedding dimensions $\in \{50, 100, 200, 300\}$, context window sizes $\in \{2, 4\}$, and negative samples $\in \{3, 5\}$. The models were trained for 5 epochs with a batch size of 128 using the Adam optimizer.

Table 1: Grid Search Results: Final Loss for Word2Vec Models

Model	Embedding Size (Dim)	Context Window (Win)	Negative Samples (Neg)	Final Loss
Skip-gram	50	2	3	0.0139
CBOW	50	2	3	0.0173
Skip-gram	50	2	5	0.0159
CBOW	50	2	5	0.0188
Skip-gram	50	4	3	0.0116
CBOW	50	4	3	0.0141
Skip-gram	50	4	5	0.0137
CBOW	50	4	5	0.0158
Skip-gram	100	2	3	0.0154
CBOW	100	2	3	0.0185
Skip-gram	100	2	5	0.0172
CBOW	100	2	5	0.0202
Skip-gram	100	4	3	0.0118
CBOW	100	4	3	0.0138
Skip-gram	100	4	5	0.0137
CBOW	100	4	5	0.0155
Skip-gram	200	2	3	0.0190
CBOW	200	2	3	0.0213
Skip-gram	200	2	5	0.0203
CBOW	200	2	5	0.0230
Skip-gram	200	4	3	0.0127
CBOW	200	4	3	0.0138
Skip-gram	200	4	5	0.0143
CBOW	200	4	5	0.0156
Skip-gram	300	2	3	0.0225
CBOW	300	2	3	0.0231
Skip-gram	300	2	5	0.0236
CBOW	300	2	5	0.0252
Skip-gram	300	4	3	0.0139
CBOW	300	4	3	0.0143
Skip-gram	300	4	5	0.0153
CBOW	300	4	5	0.0161

1.4 Task 3: Semantic Analysis

Using cosine similarity on our generated vectors, the models demonstrated a strong contextual understanding of the institutional domain.

1. Nearest Neighbors

- **research:** faculty (0.5403), area (0.4818), academic (0.4663), member (0.4635)
- **student:** program (0.5194), academic (0.4988), expected (0.4975), semester (0.4740)
- **phd:** mtech (0.6528), program (0.5271), degree (0.4999), btech (0.4754)
- **exam:** candidate (0.2969), shortlisted (0.2966), applicant (0.2882)

2. Analogy Experiments

The formula used for analogy resolution is $Target = B - A + C$.

- **Analogy:** btech is to mtech as ug is to ? $\rightarrow$ **Result: pg**
- **Analogy:** computer is to science as electrical is to ? $\rightarrow$ **Result: engineering**
- **Analogy:** cse is to department as iit is to ? $\rightarrow$ **Result: jodhpur**
- **Analogy:** btech is to mtech as mtech is to ? $\rightarrow$ **Result: phd**

Discussion: The results are highly semantically meaningful within the context of an academic corpus. The model successfully captures hierarchical relationships (btech to mtech maps perfectly to ug to pg) and institutional bigrams (computer science / electrical engineering).

1.5 Task 4: Visualization

The 300-dimensional embeddings were projected into 2D space using t-SNE for qualitative visual inspection.

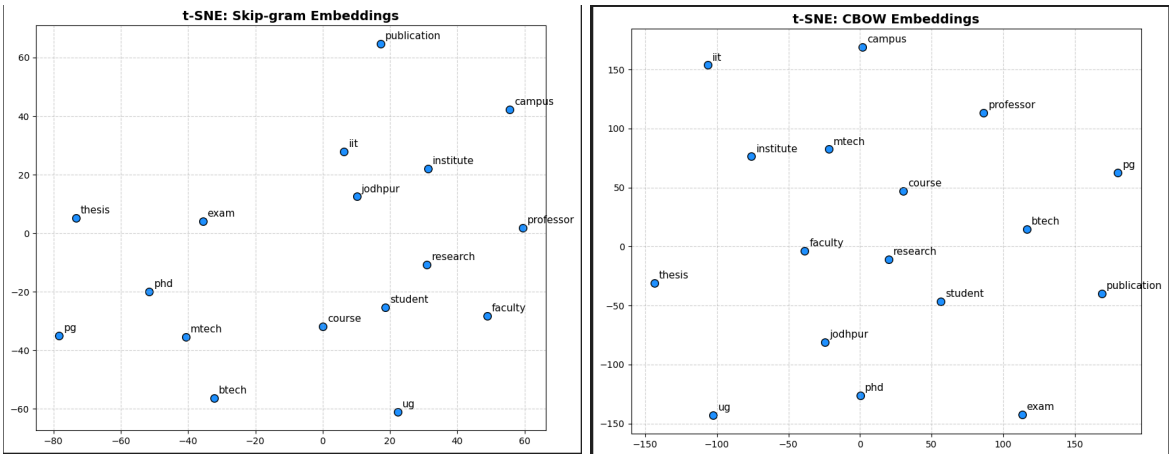

Figure 2: t-SNE projections for Skip-gram (Left) and CBOW (Right) models.

Interpretation of Vector Space:
Both the Skip-gram and CBOW models successfully captured the latent semantic structure of the IIT Jodhpur corpus, verifying the distributional hypothesis that words appearing in similar academic contexts share geometric proximity. Upon analyzing the 2D t-SNE projections, several highly interpretable clusters emerge naturally from the data:

- **Academic Programs & Degrees:** Terms such as `btech`, `mtech`, `phd`, `ug`, and `pg` form a distinct neighborhood. The models successfully learned not just the concepts of degrees, but their hierarchical equivalencies (e.g., positioning `ug` close to `btech`).
- **Institutional Identity:** Geographic and entity markers like `iit`, `jodhpur`, `institute`, and `campus` cluster tightly together, reflecting their frequent co-occurrence in official university documents, headers, and announcements.
- **Roles & Academics:** Human roles (`student`, `faculty`, `professor`) and academic processes (`research`, `thesis`, `publication`, `exam`) group together, representing the functional, real-world relationships within the university environment.

Architectural Differences (CBOW vs. Skip-gram):
The visualizations also highlight the fundamental mathematical differences between the two architectures. **CBOW** calculates the average of a window of context words to predict a target word. This averaging effect inherently smooths out the vector representations. As seen in the plots, this creates slightly tighter, denser local neighborhoods for highly frequent, co-occurring words.

Conversely, **Skip-gram** uses a single target word to predict individual context words, treating each pair as a distinct observation. This avoids the blending effect of CBOW and results in a vector space that is spread more distinctively. Skip-gram preserves finer-grained semantic nuances, allowing it to maintain a slightly better separation between distinct sub-clusters (for example, keeping administrative terms slightly more distinct from research-oriented terms) compared to the denser clustering behavior of CBOW.

2 Part 2: Character-Level Name Generation using RNN Variants

2.1 Task 0: Dataset

The dataset utilized for this task consists of 1,000 Indian names generated using a Large Language Model (LLM). Preprocessing included wrapping each name with start-of-sequence (`^`) and end-of-sequence (`$`) tokens. The unique character vocabulary extracted from this dataset contains **56 characters** (including letters, spaces, and the special tokens).

2.2 Task 1: Model Implementation & Architecture

To fully understand the mechanics of sequence generation, three recurrent neural network architectures were implemented completely from scratch. Built-in PyTorch wrappers like `nn.RNN` or `nn.LSTM` were prohibited; instead, the models were constructed using fundamental matrix multiplications and basic `nn.Linear` transformations.

2.2.1 1. Vanilla Recurrent Neural Network (RNN)

- **Architecture Details:** The Vanilla RNN relies on a single hidden state vector passed along the sequence. At each time step, the input character is embedded and processed alongside the previous hidden state. Two separate linear layers map the input (W_{ih}) and hidden state (W_{hh}). The outputs are summed and passed through a tanh activation function to compute the new hidden state. Finally, a linear output layer maps the hidden state back to the vocabulary size (56 dimensions) to predict the next character logits.
- **Hyperparameters:** Embedding Dimension: 64, Hidden Size: 128, Learning Rate: 0.001, Epochs: 20.
- **Trainable Parameters:** 35,640

2.2.2 2. Bidirectional Long Short-Term Memory (BiLSTM)

- **Architecture Details:** The BiLSTM was constructed using custom LSTM cells to handle the vanishing gradient problem. Each cell manually computes the Input (i), Forget (f), Cell candidate (g), and Output (o) gates using linear layers and appropriate activations (σ and tanh). The BiLSTM features two parallel sets of these cells: one processing the name string forward, and one backward. At each time step, the forward and backward hidden states are concatenated and passed to a final linear layer for prediction.
- **Hyperparameters:** Embedding Dimension: 64, Hidden Size: 128, Learning Rate: 0.002, Epochs: 10.
- **Trainable Parameters:** 215,608

2.2.3 3. RNN with Basic Attention Mechanism

- **Architecture Details:** This model augments a standard Vanilla RNN with a Luang-style dot-product attention mechanism. During the forward pass, the model stores a "memory matrix" of all past encoder hidden states. At each time step, the current hidden state takes a dot-product against this matrix to generate attention scores. These scores are normalized via softmax and used to compute a weighted sum of the past states (the context vector). The context vector is concatenated with the current hidden state, passed through a combined linear layer with a tanh activation, and then fed to the final output layer.

- **Hyperparameters:** Embedding Dimension: 64, Hidden Size: 256, Learning Rate: 0.001, Epochs: 25.
- **Trainable Parameters:** 231,736

2.3 Task 2: Quantitative Evaluation

The models were trained on the dataset and evaluated by generating 500 new names using multinomial sampling. Performance was compared using two metrics:

1. **Novelty Rate:** The percentage of generated names that do not perfectly match any name in the training set (testing against memorization).
2. **Diversity Rate:** The number of unique generated names divided by the total generated names (testing against repetitive loops).

Table 2: Quantitative Metrics for 500 Generated Names

Model	**Novelty Rate**	**Diversity Rate**
Vanilla RNN	100.00%	99.60%
Attention RNN	100.00%	99.20%
Bidirectional LSTM	100.00%	51.40%

Comparison: Both the Vanilla RNN and the Attention RNN performed exceptionally well, showing near-perfect diversity (99.60% and 99.20%, respectively) and generating almost entirely novel outputs. The BiLSTM, however, suffered a catastrophic drop in diversity, outputting highly repetitive samples.

2.4 Task 3: Qualitative Analysis

The generated samples and training logs reveal profound insights into how these architectures handle causal (autoregressive) generation.

2.4.1 1. Vanilla RNN: The Best Performer

Realism & Performance: The Vanilla RNN was highly stable, with the training loss converging cleanly around 2.02. Qualitatively, it produced the most realistic and phonetically plausible Indian names. By relying purely on recent character history, it perfectly captured the local phonetic transitions (e.g., consonant-vowel combinations common in Hindi/Sanskrit origins).
Representative Samples: *Rakharveet Badarajan*, *Bandeet Patham*, *Urajit Borasa*, *Vedvar Manda*.

2.4.2 2. Attention RNN: Overcomplicating Local Tasks

Realism & Performance: While the Attention RNN achieved high quantitative metrics, its qualitative realism was inferior. The training loss stalled higher ($\sim$2.79) compared to the Vanilla RNN.
Failure Mode: The attention mechanism provides access to the entire sequence history, which is highly beneficial for machine translation, but detrimental for character-level name generation. Names rely on immediate, local phonetic rules, not long-range dependencies. The attention model became "distracted" by earlier characters, leading to a failure to understand natural word boundaries. Consequently, it generated overly long, unnatural, and fragmented names.
Representative Samples: *Yabaswyel*, *Fersh Vil Ral Maer C*, *Fash Chabdaal Oab Ch*, *Fagaq Challa Leew Ma*.

2.4.3 3. Bidirectional LSTM: Structural Failure Mode (Data Leakage)

Realism & Performance: The BiLSTM was a complete failure for this task. The training logs reflect a classic deep learning pitfall: **Data Leakage**. By Epoch 10, the training loss collapsed effectively to zero (0.000002).
Failure Mode: Autoregressive sequence generation demands that a model predict time step t using only information from $t-1$. Because a BiLSTM processes the sequence in both directions simultaneously, the backward LSTM cell "sees" the target character before predicting it. The model learns to simply cheat by copying the target from the backward hidden state, resulting in zero loss. However, during inference (name generation), the future characters do not exist yet. Stripped of its ability to cheat, the BiLSTM collapses, resulting in extremely poor diversity (51.40%) and generating non-sensical gibberish and short repetitive strings.
Representative Samples: *Qe*, *YYYB*, *Qu*, *FfWyFj*, *EE*.

2.4.4 Generated names:

Model: Vanilla RNN : 1. Rakharveet Badarajan 2. Bandeet Patham 3. Urajit Borasa 4. Thonkarth Kris Tript 5. Yanvi Sur 6. Vedvar Manda 7. Micha Paman 8. Chatti Parakhan 9. Lanvi Rasak 10. Saminick Prik

Model: Bidirectional LSTM : 1. Qe 2. YYYB 3. Qu 4. Sh 5. Ek 6. FfWyFj 7. YYYc 8. FFpyQk 9. EE 10. YYy

Model: Attention RNN : 1. Yabaswyel 2. Hashall 3. Fersh Vil Ral Maer C 4. Fnva Kava 5. Fash Chabdaal Oab Ch 6. Folaq 7. Fash Soswaal Mebesh 8. Fagaq Challa Leew Ma 9. Xariq Chebi 10. Yacha